

파이썬 컨테이너 자료구조 완벽 가이드

기본 4대 데이터 타입(List, Tuple, Dictionary, Set)의 특징 및 활용 전략

파이썬에서 **컨테이너(Container)**란 여러 개의 객체를 하나의 단위로 묶어서 관리할 수 있는 자료구조를 말합니다. 파이썬은 내부적으로 데이터의 멤버십 테스트(in 연산자)를 지원하는 객체를 뜻하며, 목적과 데이터의 특성에 따라 크게 4가지 기본 내장 컨테이너를 제공합니다.

1. 핵심 컨테이너 한눈에 비교

자료구조	선언 기호	순서 여부 (Ordered)	수정 가능 여부 (Mutable)	중복 허용	주요 용도
List (리스트)	[]	O	Mutable	O	순차적인 데이터 보관, 일반적 용도
Tuple (튜플)	()	O	Immutable	O	수정되면 안 되는 고정 데이터, 다중 반환 값
Dictionary (딕셔너리)	{Key: Value}	O (3.7+)	Mutable	Key 불가, Value 가능	키-값 쌍의 매핑 구조, 빠른 탐색
Set (집합)	{ }	X	Mutable	X	중복 제거, 집합 연산 (교/합/차집합)

2. 컨테이너별 상세 분석 및 핵심 문법

■ List (리스트) - 가장 범용적인 가변 시퀀스

리스트는 순서가 보장되며 데이터를 자유롭게 추가, 수정, 삭제할 수 있는 가장 대표적인 자료구조입니다. 내부적으로 동적 배열(Dynamic Array) 구조로 구현되어 있어 인덱스를 통한 직접 접근이 매우 빠릅니다.

```
# 리스트 기본 선언 및 조작
fruits = ["사과", "바나나"]
fruits.append("체리")      # 맨 뒤에 추가 -> ["사과", "바나나", "체리"]
fruits.insert(1, "망고")   # 특정 위치 삽입 -> ["사과", "망고", "바나나", "체리"]
fruits.remove("바나나")    # 값으로 삭제 -> ["사과", "망고", "체리"]

# 인덱싱과 슬라이싱
print(fruits[0])           # '사과' (첫 번째 요소)
print(fruits[-1])          # '체리' (마지막 요소)
print(fruits[0:2])         # ['사과', '망고'] (0번부터 1번까지 잘라내기)
```

■ Tuple (튜플) - 안전성이 보장된 불변 시퀀스

튜플은 리스트와 거의 동일하지만, **한 번 생성하면 내부 값을 절대 변경할 수 없다(Immutable)**는 결정적인 차이가 있습니다. 값이 변하지 않기 때문에 리스트보다 메모리를 적게 소모하며 프로그래밍의 안전성을 높여줍니다.

```
# 튜플 기본 선언
point = (10, 20)
user_info = ("홍길동", 30, "서울")

# 원소가 하나인 튜플을 만들 때는 반드시 심표(,)를 붙여야 함
single_element = (5,)

# 튜플 활용: 다중 반환 및 구조 분해 (Unpacking)
x, y = point
print(f"x 좌표: {x}, y 좌표: {y}") # x 좌표: 10, y 좌표: 20

# 에러 발생 케이스 (수정 불가)
# point[0] = 50 -> TypeError 발생!
```

💡 리스트 vs 튜플 선택 기준

프로그램이 실행되는 동안 데이터의 크기나 내용이 계속 바뀌어야 한다면 **List**를 사용하고, 데이터베이스 레코드나 좌표값처럼 한 번 정의된 후 데이터 구조가 고정되어야 한다면 함수 간 데이터 전달 시의 안전성을 위해 **Tuple**을 사용하는 것이 정석입니다.

■ Dictionary (딕셔너리) - 키(Key)와 값(Value)의 매핑

딕셔너리는 사전식 구조로, 단어(Key)와 뜻(Value)이 짝을 이루는 형태입니다. 내부적으로 해시 테이블(Hash Table)을 사용하여 데이터의 양이 아무리 많아도 Key를 통해 값을 찾는 속도가 초고속($O(1)$)으로 유지됩니다.

```

# 딕셔너리 기본 선언 및 조작
user = {
    "name": "이순신",
    "age": 45,
    "city": "부산"
}

# 데이터 접근 및 추가
print(user["name"])          # '이순신'
user["email"] = "lee@com"    # 새로운 키-값 추가
user["age"] = 46              # 기존 값 수정

# 안전하게 값 가져오기 (Key가 없을 때 에러 방지)
job = user.get("job", "무직") # 'job' 키가 없으면 기본값 '무직' 반환

# 키, 값 분리 조회
print(user.keys())           # dict_keys(['name', 'age', 'city', 'email'])
print(user.values())          # dict_values(['이순신', 46, '부산', 'lee@com'])

```

■ Set (집합) - 중복을 허용하지 않는 고유 데이터 바구니

집합은 수학의 집합 개념을 코드로 구현한 자료구조입니다. 순서가 없기 때문에 인덱싱([0])이 불가능하지만, **중복된 데이터를 완전히 배제**하며 고유한 값만 남겨둡니다. 교집합, 합집합 등 집합 연산에 특화되어 있습니다.

```

# 집합 선언 및 중복 제거
numbers = {1, 2, 2, 3, 3, 3}
print(numbers)          # {1, 2, 3} (중복 자동 제거됨)

# 리스트의 중복 제거용으로 자주 쓰임
raw_list = ["A", "B", "A", "C", "B"]
unique_list = list(set(raw_list)) # ['A', 'B', 'C']

# 집합 연산
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

print(set_a & set_b)      # 교집합: {3, 4}
print(set_a | set_b)      # 합집합: {1, 2, 3, 4, 5, 6}
print(set_a - set_b)      # 차집합: {1, 2}

```

3. 컨테이너 활용 필수 내장 함수

모든 컨테이너 객체에 공통적 혹은 유연하게 적용되어 데이터 전처리를 도와주는 핵심 내장 함수들입니다.

```
# len() : 데이터의 총 개수 확인
nums = [10, 20, 30]
print(len(nums)) # 3

# sum() : 숫자형 데이터의 총합
print(sum(nums)) # 60

# max(), min() : 최댓값, 최솟값
print(max(nums)) # 30
print(min(nums)) # 10
```

```
# sorted() : 정렬된 새로운 리스트 반환
unordered = [4, 1, 3, 2]
ordered = sorted(unordered)
print(ordered) # [1, 2, 3, 4]

# in 연산자 : 멤버십 테스트
fruits = ["apple", "banana"]
print("apple" in fruits) # True
print("grape" in fruits) # False
```

문서 요약 정리

데이터를 다룰 때 순서가 중요하고 자주 바뀐다면 **List**를, 고정된 데이터 값의 묶음이라면 **Tuple**을, 이름표(**Key**)를 붙여 빠르게 매핑해야 한다면 **Dictionary**를, 중복을 없애고 유일한 값만 관리하려면 **Set**을 선택하는 것이 가장 효율적인 알고리즘 설계의 시작입니다.